



Contents lists available at ScienceDirect

Blockchain: Research and Applications

journal homepage: www.journals.elsevier.com/blockchain-research-and-applications

Research Article

SoK: On the security of non-fungible tokens

Kai Ma^{a,1}, Jintao Huang^{a,1}, Ningyu He^{b,1}, Zhuo Wang^a, Haoyu Wang^{a,*}^a School of Cyber Science and Engineering, Huazhong University of Science and Technology, Wuhan 430074, China^b School of Computer Science, Peking University, Beijing 100871, China

ARTICLE INFO

Keywords:

Blockchain

Non-fungible token (NFT)

Security

Smart contract

ABSTRACT

Non-Fungible Tokens (NFTs) drive the prosperity of the Web3 ecosystem. By May 2024, the total market value of NFT projects reached approximately \$69 billion. Accompanying the success of NFTs are various security issues, i.e., attacks and scams are prevalent in the ecosystem. While NFTs have attracted significant attention from both industry and academia, there is a lack of understanding of the kinds of NFT security issues. The discovery, in-depth analysis, and systematic categorization of these security issues are of significant importance for the prosperous development of the NFT ecosystem. To fill this gap, we perform a systematic literature review related to NFT security and identify 176 incidents from 248 security reports and 35 academic papers until May 1st, 2024. Through manual analysis of the compiled security incidents, we classify them into 12 major categories. Then, we explore potential solutions and mitigation strategies. Drawing from these analyses, we establish the first NFT security reference frame. In addition, we extract the characteristics of NFT security issues, i.e., the prevalence, severity, and intractability. We highlight the gap between industry and academia for NFT security and provide further research directions for the community. This paper, as the first Systematization of Knowledge (SoK) of NFT security, systematically explores security issues within the NFT ecosystem, shedding light on their root causes, real-world attacks, and potential ways to address them. Our findings will contribute to future research on NFT security.

1. Introduction

Since the beginning of 2021, Non-Fungible Tokens (NFTs), a groundbreaking form of currency, have surged in popularity [1]. The most notable feature of NFT is its uniqueness. In other words, there is a cryptocurrency-based algorithm to guarantee that it is impossible to counterfeit an existing NFT. Taking advantage of this characteristic, as well as the anonymity and traceability of blockchain techniques, users can bind an NFT to real-world assets, such as images, videos, and even in-game items, through a Uniform Resource Identifier (URI) [2]. To this end, directly exchanging NFTs among users has greatly reduced the transaction costs of goods. Currently, many NFT projects have received considerable investment. For example, one NFT project called Bored Ape Yacht Club (BAYC) [3] has reached a total volume of over 1.5 million ETH [4].

However, alongside their increasing popularity, the whole NFT ecosystem is under numerous security threats that have resulted in millions of dollars in losses for investors and project owners. For example, in April 2022, an individual lost his BAYC NFT worth over \$3 million

in a fraudulent swap transaction [5]. Similarly, in May 2022, a family was scammed out of \$1.4 million by a perpetrator who sold them a counterfeit Moonbirds token [6], which had taken them years to accumulate. In September 2022, hackers stole Ethereum valued at \$185,000 from Bill Murray after he auctioned an NFT for charity [7]. According to a research report [8], there are currently more than 200 attack events against the NFT ecosystem, causing a total of more than \$88 million in economic losses. More importantly, the frequency of these attacks continues to increase year by year.

In recent years, several studies on NFT security have been conducted. For example, Das et al. [9] conducted the first systematic analysis of NFT security. Stöger et al. [10] found that almost every interaction of a user with a centralized entity can be exploited to hijack NFTs or cryptocurrencies from the user. However, they focused only on NFT market manipulation in the corresponding NFT projects, without paying attention to other components of the NFT ecosystem, like NFT smart contracts, which have also been exploited by adversaries to gain illegal profits.

* Corresponding author.

E-mail address: haoyuwang@hust.edu.cn (H. Wang).¹ Equal contribution.<https://doi.org/10.1016/j.bcr.2024.100268>

Received 19 May 2024; Received in revised form 6 September 2024; Accepted 25 December 2024

Available online 2 January 2025

2096-7209/© 2025 THE AUTHORS. Published by Elsevier B.V. on behalf of Zhejiang University Press. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

To fill this gap, we conduct the first Systematization of Knowledge (SoK) on the security side of the whole NFT ecosystem. Against the NFT ecosystem, our goal is to summarize all happened attack events, uncover all their root causes, and propose corresponding best practices for developers and investors.

In this SoK, we first construct an NFT security reference frame in Section 3. We collect data from both security reports and academic papers and then extract the incidents and security issues. Next, based on the reference frame, we dive deep into the contract layer in Section 4, the market layer in Section 5, and the auxiliary service layer in Section 6. Specifically, we further discuss the characteristics, current research, and potential detection/defense methods of the extracted security issues. We give our key findings and provide a discussion as well as potential future research directions in Section 7. Sections 8 and 9 present related work and the conclusion, respectively.

Our contributions and key findings are as follows:

- We conduct the first SoK on NFT security, encompassing both prominent and lesser-known security issues. Specifically, we compile the first NFT security reference frame, with 176 incidents belonging to 12 major categories, which serves as a general model for NFT security.
- We extract the characteristics of NFT security issues. We demystify these NFT security issues from characteristics including the prevalence, severity, and intractability. These characteristics serve as a compelling motivation to delve into these security issues.
- We point out the huge gap between industry and academia. Our results show that current research on NFT security addresses only a limited portion of the NFT security issues that have been discovered by the industry.

We release our crafted dataset at <https://nftsoK.github.io/>.

2. Background

2.1. Blockchain and cryptocurrency

Blockchain implements a distributed ledger that uses cryptography to securely link growing lists of records, i.e., blocks [11], together. Each block contains a set of transactions generated by accounts [12], i.e., the participants in the blockchain network. Blocks are continuously added to the chain to form an immutable record of all past transactions, creating what is known as the process of mining [13]. The chosen block is determined by consensus protocols [14] such as Proof of Work (PoW) [15] or Proof of Stake (PoS) [16]. Bitcoin [17] was the first digital token implemented by blockchain technology. Ethereum first introduced the concept of smart contracts [18] in 2014, which are sets of codes that run on the blockchain. Transactions can be initiated between accounts or even contracts to achieve complex functionalities, such as transferring funds to another account or conducting calculations.

Smart contracts enable the creation of various digital tokens, extending beyond Bitcoin, which includes both fungible tokens and NFTs. Typically, these digital tokens are developed following specific protocols, such as the Ethereum Request for Comments (ERC) in the case of Ethereum. In Ethereum, fungible tokens are established under the ERC-20 standard, while NFTs are primarily implemented through the ERC-721 and ERC-1155 standards.

Specifically, ERC-721 [19] is a standard protocol designed for NFTs where each token is bound to a single piece of digital content, e.g., art, music, and video. ERC-721 tokens cannot be divided into smaller units and must be traded as whole entities. However, it has limitations, e.g., difficulties in representing repeated artwork and bulk transfers of a large number of tokens at once. To address these issues, ERC-1155 [20] was introduced, which allows multiple tokens to be associated with specific content, allowing all token holders to share access to that particular content. It also facilitates multi-token operations that enable the transfer

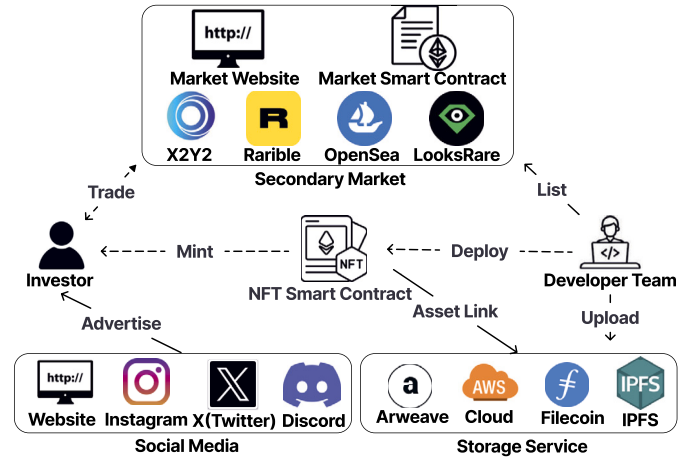


Fig. 1. Non-Fungible Token (NFT) life cycle. It shows the various stages of an NFT and the people taking part. The dashed lines stand for actions recorded on the blockchain, and the solid lines represent off-chain activities not recorded on the chain.

of multiple tokens in a single transaction. Other standards, like Metaplex in Solana [21,22] and BEP-721 in Binance [23,24], also support the nature of NFTs.

2.2. Decentralized application (DApp)

Taking advantage of the interactions among smart contracts, developers can achieve complex functionalities in a DApp. DApps can operate on blockchain networks, ensuring data immutability, and guaranteeing the authenticity and reliability of information. Moreover, after porting traditional finance applications on decentralized blockchains, decentralized finance (DeFi) gradually emerges in 2017 [25]. DeFi relies on smart contracts and decentralized networks to offer services like lending, borrowing, trading, and earning interest on cryptocurrencies. Specifically, decentralized exchange (DEX) is a special kind of DeFi DApp that implements cryptocurrency exchange.

To alleviate the burden of managing the digital cryptocurrencies of users, wallets appear as browser plugins or standalone mobile applications [26]. They assist users in initiating cryptocurrency transactions, conducting identity verification, and interacting with smart contracts. A hot wallet is a type of wallet that remains consistently connected to the internet, offering rapid access and transaction capabilities. However, due to its online nature, it is more vulnerable to cyber threats than its offline counterpart, known as cold wallets. The private key is a randomly generated unique alphanumeric character string used to prove ownership and control over digital assets stored in the wallet.

2.3. NFT life cycle

Fig. 1 shows the overall landscape of the NFT ecosystem. Generally, it can be divided into four phases, i.e., asset creation, contract deployment, NFT minting, and market circulation.

Phase I: Asset creation. The intrinsic value of an NFT is derived primarily from its association with unique digital assets. To create an NFT project, the developer first creates a collection of digital assets, e.g., a set of images, and uploads them to off-chain storage services, where users can access these digital assets via Unique Resource Identifiers (URIs). These developers advertise their projects on public social media platforms (such as X(Twitter) [27], Discord [28], and Instagram [29]) and their official websites. Moreover, to attract investors consistently, they may also adopt some marketing strategies, such as sharing project roadmaps or highlighting the artistic and economic potential of the project.

Phase II: Contract deployment. The deployment of contracts is essentially the process of tokenizing digital assets, completing the mapping of off-chain assets to on-chain tokens. The developer deploys the smart contract under a specific NFT standard, e.g., the ERC-721 standard on Ethereum, which links the URI on-chain to the digital asset on the storage service.

Phase III: NFT minting. NFTs are created by a special event called mint. Two methods are commonly used to create NFTs for investors. First, investors can pay mint fees to smart contracts, exchanging the right to mint the token. Alternatively, the contract owner sometimes mints the tokens themselves and airdrops them to participants in the NFT ecosystem, freely distributing tokens and making them flow into the market. Each specific NFT is unique once minted by a smart contract, serving as the identification for that particular token and its associated content.

Phase IV: Market circulation. Participants in the NFT ecosystem typically perform trades of NFTs in secondary markets, i.e., special DApps focused on token trading, such as OpenSea [30], LooksRare [31], and X2Y2 [32]. After creating NFTs, developers list their tokens for sale in secondary markets [33]. Generally, these tokens are organized into collections [34] that refer to all tokens minted by the same smart contract, which helps to gather all work in the same style, create a brand, and present it to the public. Secondary markets also provide an optional verification tag [35] to help increase the credibility of the collections. As DApps, secondary markets have their smart contracts launched on the blockchain, such as Wyvern [36] or Seaport [37] from OpenSea. These smart contracts implement different functions that help users trade in a secure environment. Buyers can make offers or bids before purchasing assets. Once accepted, the smart contract on the blockchain transfers the asset to the target account. During the process, the trade fees [38] are withdrawn by the secondary market, and the creator fees [39] are withdrawn by the creator group.

3. Reference frame of NFT security

We next present our methodology to construct a reference frame, which serves as the foundation for the analysis of security issues.

3.1. Notations

We first define the following notations used in the paper:

1. **Security incidents.** An NFT security incident refers to a concrete case that can or may result in an unexpected financial loss to one or multiple of the following entities: (i) NFT holders, who own these tokens; (ii) NFT project teams, who are responsible for creating and managing NFT collections; (iii) NFT sales platforms, which facilitate the trading and exchange of NFTs. Specifically, there are two types of incidents: (i) attacks that exploit the vulnerabilities within smart contracts or the defects within the market and auxiliary services and (ii) accidents that do not explicitly involve proactive adversaries but still cause damage to relevant parties. For example, asset inaccessibility (see Section 6.3) is labeled as an accident. This is because even if attackers or scammers do not intentionally take the corresponding actions, they can still result in significant harm to the affected individuals in the community.
2. **Security issues and sub-issues.** A security issue refers to an abstraction of a set of incidents that possess similar causes and processes. If necessary, an issue can be further divided into sub-issues. For example, the reentrancy security issue consists of two sub-issues, i.e., single-function and cross-function (see Section 4.1.1). Similarly, each sub-issue is still composed of several incidents with similar root causes. Thus, security issues, sub-issues, and incidents form a multi-level inclusion relationship.
3. **Category.** Category refers to the abstract representation for the taxonomy and description of a group of related security issues.

4. **Layer.** A layer is composed of components in the NFT ecosystem and refers to an abstract location where a set of security issues occur.

3.2. Data collection

3.2.1. Scope

To conduct a comprehensive analysis, we consider two types of materials as sources, i.e., academic papers and security reports. Specifically, academic papers generally conduct systematic research on security issues, which allows them to go beyond reported security incidents and propose more general detection methods. Even further, they sometimes suggest defense mechanisms. However, publishing academic papers often requires a significant amount of time for research, which means that academic papers on security issues tend to lag behind industry technical reports.

Given that NFTs are a relatively new concept, some papers may still be in the pipeline and are only available on arXiv for now. Therefore, we also included arXiv articles in our collection.

Thus, we collected security reports published by flagship blockchain security companies because of their timeliness and professionalism. The gathering process continued until May 1st, 2024.

3.2.2. Approach

Our data collection for academic papers and security reports was divided into four steps. First, we selected a set of target sources. For security reports, we chose several reputable sources, i.e., Beosin [40], SlowMist [41], Blocksec [42], Certik [43], DeFiHackLabs [44], and De.Fi [45]. Note that if these reports often cite other security reports, they are also included in our database. For academic papers, we performed a thorough search on academic databases, i.e., Google Scholar, IEEE, and ACM, with the keywords NFT and DeFi. Second, we collected all related materials from these sources. Then, we employed a rigorous review process. We engaged two Ph.D. students with expertise in the blockchain and security domains to meticulously review the collected materials. To ensure accuracy and eliminate potential biases, a paper was retained for our study only if both reviewers independently recognized its relevance to NFT security incidents or issues. In the final step, we recognized that relying solely on our initial search terms could lead to the overlooking of relevant papers. Thus, we took all identified security issues as keywords to perform a comprehensive search on these databases again, like specifically searching reentrancy. Note that while these articles may not be exclusively about NFTs, the research approaches for these general issues can be applied to study those related to NFT issues. This ensures that our collected academic papers and security reports comprehensively cover issues strongly related to the NFT ecosystem.

3.2.3. Dataset overview

Consequently, we identified 248 security reports and 35 academic papers. Certain incidents may be reported multiple times. After manually removing duplicates, 176 incidents from both security reports and academic papers remained. The dataset is open-sourced at <https://nft-sok.github.io/>.

3.3. NFT ecosystem layering

Building on the life cycle of NFTs depicted in Section 2.3 and widely adopted frames of blockchain security from existing work [46], we define three distinct layers for the whole NFT ecosystem, i.e., the contract layer, market layer, and auxiliary service layer. We classify all security issues according to the specific location where they occur. Specifically, the contract layer comprises all security issues in contracts, resulting from code weaknesses that attackers could exploit. Note that we focus on not only the Ethereum blockchain but also other blockchain platforms that can mint NFTs, like Solana. The market layer is composed of security issues that happened on NFT markets. This involves examining

Table 1

All 16 kinds of security issues within the contract layer, market layer, and auxiliary service layer.

Layer	Category	Security issue	Sub-issue	IEI	BP
Contract layer	Unsafe external call (12)	Reentrancy	Single-function reentrancy	●	○
			Cross-function reentrancy	○	○
	Insufficient access control (7)	Denial of Service (DoS)	-	●	●
		Ownership verification issue	-	○	○
		Function visibility issue	-	○	○
	Improper business logic (16)	Conditional expression bug	Lack of conditional expression	○	○
			Erroneous conditional expression	○	○
Bad randomness (2)	Variable-related bug	-	○	○	
	Bad randomness	-	●	●	
Market layer	Wash trading (2)	Wash trading	-	●	○
	Arbitrage (3)	Arbitrage	Collection offer arbitrage	●	○
			Flashloan-based arbitrage	○	○
	Rug pull (47)	Rug pull	-	●	●
	Copyright theft (3)	Sleepmint	-	●	●
		Counterfeiting NFT	Name counterfeiting	●	○
			Image counterfeiting	●	○
Auxiliary service layer	Website exploitation (10)	Website exploitation	Malicious script upload	○	○
			API exploitation	○	○
			User Interface (UI)-based fraud	○	○
			Distributed DoS (DDoS) attack	○	○
	NFT-themed phishing (62)	NFT-themed phishing	Website phishing	●	●
			Phishing trojans	○	○
			Spam NFTs	●	○
			Hacking social media	○	○
	Asset-related issue (1)	Asset-related issue	Asset inaccessibility	●	○
			Metadata tampering	●	○
Private key compromised (11)	Private key compromised	-	○	○	

The number inside the parentheses indicates the quantity of collected incidents related to a specific security issue. Note that there are no incidents related to asset-related issues. This is because both sub-issues were identified solely from academic papers, and there is no technical report discussing them. We use different symbols to mark the status of current research on Identifying Existing Incidents (IEIs) and Best Practice (BP) topics. In particular, “●” indicates that a topic has been studied before, and “○” means that it has not been technically explored in previous research.

unusual and harmful trades, as well as malicious actions affecting the sellers and buyers or even everyone involved in the market. The auxiliary service layer is defined as infrastructures that help in the regular workflow of NFT, such as social media or digital wallets. Therefore, we classify any security issues related to these infrastructures in this layer.

Table 1 presents all 16 security issues that we are concerned with, as well as the category and layer to which they belong. For each security issue, we further summarize the types of existing exploitation tactics. Moreover, we also conduct a comprehensive survey on current methods that can identify existing incidents and the best practices that can be adopted for each security issue. Based on these materials, we discuss the answers to the following questions:

1. What constitutes a security incident for a security issue/sub-issue?
2. How to identify security incidents targeting a specific issue after occurrence?
3. What best practices can be adopted to prevent losses before incidents occur?

4. Contract layer

We have summarized 7 kinds of security issues in the contract layer.

4.1. Unsafe external call

During the lifecycle of a smart contract, it may invoke functions deployed by other accounts or even contracts. Since these functions are out of the developer's control, they may perform malicious operations, leading to unsafe external calls.

```

1 function mintNFT(...) public payable {
2   for(uint i = 0; i < _numOfTokens; i++) {
3     _safeMint(msg.sender, totalSupply() + 1);
4   }
5 }
6 function _safeMint(...) internal virtual {
7   require(
8     _checkOnERC721Received(address(0), to, tokenId, _data),
9     "ERC721: transfer to non ERC721Receiver implementer" );
10 }

```

Listing 1: Case of single-function reentrancy: HyberBear.

4.1.1. Reentrancy

Reentrancy allows attackers to make multiple calls and callbacks to gain additional profits, which goes against the original intent of the function. According to the number of functions involved during the attack, two types of reentrancy have been observed in the NFT ecosystem.

Sub-issue I: Single-function reentrancy. Single-function reentrancy refers to both the caller of an external call and the callee of the corresponding callback being the same function. Internal functions are used as functional modules of public functions, and they are considered as the same function. For example, the `mintNFT` function of HyberBear has been exploited [47], as shown in Listing 1. After minting a token, `_checkOnERC721Received()` (L7²) calls the designated function, where attackers can reinvoke the `mintNFT` function of HyberBear to repeat the mint process to gain profits.

Sub-issue II: Cross-function reentrancy. Rather than entering the identical function, attackers may enter a different function that relies on the same states to achieve the reentrancy attack. The OMNI is exploitable to this vulnerability [48–50], and its implementation is shown in Listing 2. The `executeWithdrawERC721` function would externally

² L7 refers to the 7th line, we adopt this notation in the following.


```

1 function executeWithdrawERC721(...) external returns (uint256) {
2     bool withdrawingAllCollateral = INTOKEN(reserveCache.
        xTokenAddress).burn(...);
3     if (userConfig.isBorrowingAny())
4     ...
5 }
6 function burn(...) external virtual override onlyPool returns (
    bool) {
7     IERC721(_underlyingAsset).safeTransferFrom
8 }
9 function executeERC721LiquidationCall(...) external {
10     userConfig.setBorrowing(
11         liquidationAssetReserve.id, false);
12 }

```

Listing 2: Case of cross-function reentrancy: OMNI.

call the burn function (L6) to burn NFTs used as collateral tokens, subsequently returning the pledged assets to the user (L1). However, the burn function involves a callback mechanism (L7), thus providing an opportunity for attackers. During the callback process, attackers enter a different function executeERC721LiquidationCall (L9). The shared state variable indicating borrowing status is modified to False (L10). Finally, the flow returns to the initial executeWithdrawERC721 function, bypassing the check (L3), and the attack is successfully executed. The attacker does not need to repay the loan.

Identifying existing incidents. The key characteristic of exploitation on reentrancy vulnerability is the presence of a circular call in the traces. Thus, we can identify existing attacks by identifying circular call in transaction traces, where functions are nodes and invocations are directed edges. Moreover, to increase the accuracy, additional rules can be incorporated. For instance, it is possible to check whether the initiator of the circular call makes profits or if there is a decrease in the Total Value Locked (TVL) of the contract, which is a vital metric to show the value of one project.

Best practice. The fundamental cause of reentrancy is the untimely update of state variables. The speciality of NFT contracts lies in their implicit calls to the onERC721Received and onERC1155Received functions. A contract must implement one of these to receive an NFT, helping prevent the NFT from being stuck in a contract that cannot handle NFTs. Developers should pay attention to this mechanism and use the community-endorsed checks-effects-interactions pattern [51], which advocates that calling external contracts should always be the final step.

4.1.2. Denial of Service

Attackers can take advantage of external calls and deliberately make those calls fail, preventing the contract function from executing successfully and directly leading to a Denial of Service (DoS). This exploitation specifically leverages mechanisms such as revert or out-of-gas conditions. The revert mechanism allows a contract to stop execution and revert any changes while ensuring that all gas is consumed, effectively blocking the completion of the function. On the other hand, the out-of-gas condition occurs when a transaction consumes all allocated gas before completion, forcefully stopping the execution due to insufficient gas, which can be strategically induced by malicious entities to disrupt contract operations. As shown in Listing 3, it is a DoS vulnerability discovered in Akutar [52]. In the processRefunds() function (L1), there is a loop (L2 to L10) that refunds all users who have previously placed a bid, which consequently makes an external call to each of them (L6). Since the external call lacks checks of exceptions, malicious contracts deployed by attackers may hide among users. The malicious contracts can use a revert statement within their fallback or receive functions to ensure that refund transactions always fail, leaving no one able to get their fund back.

Identifying existing incidents. DoS attacks always lead to failed transactions. Thus, the information of failed transactions can assist in identifying existing DoS attacks, like the address of callers and the reason

```

1 function processRefunds() external {
2     for (uint256 i=_refundProgress; gasUsed < 5000000 && i <
        _bidIndex; i++) {
3     ...
4     bids memory bidData = allBids[i];
5     if (refund > 0) {
6         (bool sent,) = bidData.bidder.call{value: refund}("");
7         require(sent, "Failed to refund bidder");
8     }
9     ...
10 }

```

Listing 3: Case of transaction reversion call: Akutar.

```

1 function ownerWithdrawAllTo(address toAddress) public {
2     (bool success, ) = toAddress.call{value: address(this).
        balance}("");
3     require(success, "Failed to withdraw funds.");
4 }

```

Listing 4: Case of owner verified issue: FlippazOne.

for the failure. Specifically, if an external call fails due to a revert or out-of-gas issue, it indicates a potential DoS attack.

Best practice. This issue is not exclusive to NFT contracts. Any external call lacking proper exception handling could be vulnerable to DoS attacks. NFT contract developers should avoid conducting multiple external calls in a single transaction, especially when calls are executed as part of a loop [53]. It is essential to always consider the possibility of external call failures and implement strategies to handle them. There are existing works to help developers discover potential DoSs in contracts. For example, Ghaleb et al. [54] defined the target address of a call executed in the loop's body as a sink to find data flows in which the taint source msg.sender reached the sink. Moreover, Grech et al. [55] constructed an intermediate representation based on the contract Ethereum Virtual Machine (EVM) bytecode and used Datalog-based inference rules for extracting contract properties to detect DoS vulnerability, while Liao et al. [56] used smart contract bytecode and transaction trace to exact fine-grained state dependency to detect DoS vulnerability.

4.2. Insufficient access control

Access control addresses the question of who is allowed to do this, defining who has the power to perform certain actions (e.g., minting new NFTs or withdrawing funding from a contract) within a contract [57].

4.2.1. Ownership verification issue

The contract needs to validate the address of the caller or the incoming address before executing privileged functions. For example, the token transfer function can be invoked only by the token's owner himself or others, who are authorized. However, some contracts overlook the importance of confirming the owner before such sensitive operations. The withdrawal function in a smart contract is one of the privileged functions that requires validation of the caller. A lack of verification of the caller's carrying authority can lead to financial loss. One notable example is the ownerWithdrawAllTo() function in FlippazOne, as shown in Listing 4. The function here uses the low-level call in Solidity for fund transfers but lacks verification of the incoming recipient address (toAddress) before transferring the entire balance of the current contract (address(this).balance) (L2). The vulnerability allows anyone to withdraw the entire balance within the contract.

Identifying existing incidents. When dealing with privileged functions, like token transfers and contract withdrawals, ownership verification is necessary. By actively monitoring transactions and comparing the addresses calling these functions with the actual owner addresses, potential insufficient validation issues can be identified. This process can be assisted by the interfaces defined in NFT protocols, such as

```

1 function _burn(address from, address owner, uint256 id) public {
2     require(from == owner, "not owner");
3     ...
4     emit Transfer(from, address(0), id);
5 }
6 function burn(uint256 id) external {
7     _burn(msg.sender, _ownerOf(id), id);
8 }

```

Listing 5: Case of function visibility issue: The Sandbox.

`ownerOf(uint256 _tokenId)` from the ERC721 standard interface to query token owners and `owner()` to return to the contract owner.

Best practice. We can conclude that the essence of this vulnerability is the absence of validation on `msg.sender` and incoming addresses before critical operations. The methods of existing studies can be directly adopted. For example, Yang et al. [58] determined which functions required ownership verification by reading the ERC721 standard description. After that, they used symbolic execution to detect these functions without ownership verification. Ghaleb et al. [59] used static analysis to study common access control vulnerabilities in Solidity, which can similarly be applied to NFT contracts.

4.2.2. Function visibility issue

The visibility of functions in Solidity determines the access level. Currently, there are four distinct levels, i.e., external (accessible from other contracts or accounts), internal (accessible from the contract itself and its derived contracts), public (accessible from all, i.e., combining external and internal), and private (accessible only from within the contract). Improperly applying visibility for a function may lead to security issues. A typical example is Sandbox [60], a famous project of an NFT-based game. NFT-based games integrate NFTs into digital gaming, allowing players to own, buy, and sell in-game assets as NFTs that carry real-world value. Sandbox transforms its most crucial game asset, land, into NFTs, enabling ownership and trade within the gaming ecosystem. As shown in Listing 5, the contract has one external function named `burn()` (L6) and another public function named `_burn` (L1). The `burn` function is specifically designed to burn or permanently remove land assets from the game. The visibility of `_burn` (L1) should have been set as internal and can only be called by `burn`; `burn` function can restrict parameters passed into `_burn` to finish verification. However, it is set incorrectly to public, and anyone can call this function. Although there is a check statement between `from` and `owner` (L2), the two parameters can be constructed by any caller.

Identifying existing incidents. The main purpose of internal functions is to organize and reuse code within the same contract. A common pattern is when external functions call internal ones. By using a function call graph, we can find external functions that call internal functions within the same contract. This graph shows details about the functions, such as their names, visibility, and how they are connected. With this, we can check the visibility of the called functions and spot any potential visibility mistakes.

Best practice. Both developers and users should pay attention to the visibility settings of functions, especially for internal functions. For example, developers should follow standard function naming conventions [61], like naming internal functions beginning with an underscore. Following community-recognized development guidelines helps in avoiding errors and facilitates code audits, enabling the discovery of potential issues.

4.3. Improper business logic

Security issues may arise due to mistakes made by programmers within the implementation of functions. According to our investigation, we categorize them into conditional expression bugs and variable-related bugs.

```

1 function reserveApes() public onlyOwner {
2     uint supply = totalSupply();
3     uint i;
4     for (i = 0; i < 30; i++) {
5         _safeMint(msg.sender, supply + i);
6     }
7 }

```

Listing 6: Case of unlimited mint: BAYC.

4.3.1. Conditional expression bug

One of the critical purposes of conditional expressions is to protect critical functions from being arbitrarily invoked or limiting the number of certain variables. A buggy conditional expression always leads to unexpected behaviors. According to our dataset, two types exist, i.e., lack of conditional expression and erroneous conditional expression.

Sub-issue I: Lack of conditional expression. When conditional expressions are missed, some functions or statements that should be executed only a limited number of times will be called without restriction, which is likely to bring unexpected catastrophic consequences. The well-known NFT collection, BAYC, serves as a classic example of lacking essential condition checks [62]. A limit on the number of NFTs that can be minted is crucial for ensuring the rarity and value of NFT. Therefore, when minting a token, the contract needs to check whether the current supply is over the maximum supply of the collection. However, as demonstrated in Listing 6, BAYC does not have a necessary assessment of the total supply before minting one token (before L5), leading to the possibility of minting more tokens than the initially configured amount.

Sub-issue II: Erroneous conditional expression. In addition to a lack of conditional expression, there are instances where programmers might write the conditional expression incorrectly. This can lead to significant issues, such as locking funds. Erroneous conditional expressions leading to unintended behavior, such as locked funds, are exemplified by the Akutar project [52,63]. As shown in Listing 7, the second requirement (L3) has a mistake, which should compare the `refundProgress` with `_bidIndex` instead of `totalBids`. Consequently, the requirement will never be satisfied. Over 11,500 ETH is locked in the contract forever since it can never withdraw the balance (L6).

Identifying existing incidents. The conditional expression can be regarded to the verification of state variables within a function. In the EVM, the verification process involves accessing specific slots and comparing the retrieved values. If there is no access to storage slots containing critical variables, there might be an issue lacking conditions. Based on this characteristic, Yang et al. [58] used source code to establish a storage map and employed symbolic execution to detect the mint function that did not access the `totalSupply` storage slot, indicating a lack of `totalSupply` validation. However, it is important to note that in NFT contracts, the mint function is a standard interface with a roughly similar structure, making it easy to identify the critical variable `TotalSupply`. On the other hand, in DeFi contracts using NFTs, due to the diversity of the contracts themselves, identifying critical variables is not as straightforward. However, it is also possible to follow certain characteristics to discover potential critical variables, which are often associated with specific users (addresses) and stored within mappings.

Best practice. Developers should pay attention to the setting of conditional expressions in contract development, including the specific placement and the conditions for comparison. When it comes to operations like transfers and withdrawals, particular attention should be given to their conditional checks. Before deployment, developers can apply some software analysis methods, like symbolic execution, to discover potential issues.

4.3.2. Variable-related bug

The contract should include multiple variables for each transaction execution. However, using variables incorrectly could open up opportunities for attackers to exploit the contract. The improper use of variables

```

1 function claimProjectFunds() external onlyOwner {
2   require(block.timestamp > expiresAt, "Auction still in
   progress");
3   require(refundProgress >= totalBids, "Refunds not yet
   processed");
4   require(akuNFTs.airdropProgress() >= totalBids, "Airdrop not
   complete");
5   (bool sent, ) = project.call{value: address(this).balance}("");
6   require(sent, "Failed to withdraw");
7 }

```

Listing 7: Case of locking funds: Akutar.

```

1 struct Offer {... address seller; ... }
2 function punkNoLongerForSale(uint punkIndex) {
3   punksOfferedForSale[punkIndex] = Offer(false, punkIndex, msg.
   sender, 0, 0x0);
4 }
5 function buyPunk(uint punkIndex) payable {
6   Offer offer = punksOfferedForSale[punkIndex];
7   punkNoLongerForSale(punkIndex);
8   pendingWithdrawals[offer.seller] += msg.value;
9 }

```

Listing 8: Case of variable-related issue: CryptoPunk V1.

may lead to funds being locked in the contract forever. For example, one vulnerability was discovered in CryptoPunks V1, which was the original CryptoPunks NFT after it was launched in 2017 [64]. As shown in Listing 8, `punkNoLongerForSale` (L2) overwrites `offer.seller` for `msg.sender`, i.e., the buyer (L3). This leads to the actual seller not being able to withdraw money. This mistake almost led to the abortion of the project, but fortunately, the development team remedied it in time, which saved the most valuable NFT project today.

Identifying existing incidents. By collecting normal transactions from existing contracts and extracting the operational characteristics of normal transactions, one can establish an expected transaction model. For instance, in NFT sales, there should be a decrease in the number of holding tokens of the seller and an increase in the number of holding tokens of the buyer. By broadly establishing models for expected normal transactions, one can identify abnormal transactions that deviate from the expected pattern. For example, in the above incident, where the account balance of the seller does not increase after selling an NFT or the account balance of the buyer does not decrease, it deviates from the expected pattern and indicates a potential attack.

Best Practice. The intrinsic property of variable-related issues is unexpected value modification. Thus, developers can adopt taint analysis to trace the sources that can modify variables to uncover unexpected data flows. For instance, in the case of an incident, by tracing the potential sources for `Offer.seller`, one can uncover `msg.sender`, which indicates unexpected modification of values.

4.4. Bad randomness

Bad randomness in smart contracts refers to the random number being calculated solely based on determined seeds, which can be predicted by adversaries [65]. A buggy random number-generating algorithm can potentially be exposed by attackers, creating opportunities for exploitation. Take Meebits as an example [66], which is shown in Listing 9. To mint a token, the `_mint()` function must be called first (L1 to L6). This function, in turn, calls another function called `randomIndex()` (L7 to L11) to generate a random value, which is used as the token ID for a new NFT. However, this randomness is generated based on a set of values, including blockchain data like `nonce`, `difficulty`, and `timestamp` (L10). All of these values are visible to anyone in the blockchain network. Since Meebits NFTs have different rarities based on their Identity Documents (IDs), their prices are highly sensitive to their rarities. Consequently, this vulnerability allows malicious actors to mint NFTs with predictably high values.

```

1 function _mint(address _to, uint createdVia) internal returns (
   uint) {
2   ...
3   uint id = randomIndex();
4   emit Mint(id, _to, createdVia);
5   emit Transfer(address(0), _to, id);
6   ...
7   function randomIndex() internal returns (uint) {
8     ...
9     uint index = uint(keccak256(abi.encodePacked(
10      nonce, msg.sender, block.difficulty, block.timestamp)))
11   ...}

```

Listing 9: Case of bad randomness: Meebits.

Identifying existing incidents. If an address consistently profits from contracts relying on random numbers, it may indicate flaws in randomness, suggesting that the address might be exploited by this vulnerability. From this characteristic, we can monitor the rarity of each NFT acquired by users through mint transactions. If an address frequently acquires rare NFTs through multiple transactions, there might be potential exploitation of randomness.

Best practice. Bad randomness occurs when the random generator depends solely on determined factors, like block information (like `block.blockhash`) and environmental information (like `msg.sender`) in Ethereum. Qian et al. [65] employed taint analysis techniques to track critical variables, such as `block.blockhash` and `block.coinbase`, thereby identifying potential bad randomness in smart contracts. Developers should guarantee that there are some unpredictable seeds, like the data returned by oracles.

5. Market layer

We next present five kinds of security issues related to the NFT market.

5.1. Wash trading

Wash trading refers to a type of deceptive practice where attackers trade tokens within a group, creating an illusion of prosperity with the intention of deceiving or scamming unsuspecting investors and users. This type of activity has been deemed illegal in many countries [67]. Several real-world washing trading cases have been uncovered by the media. For example, Meebits, as one of the top collections in the NFT ecosystem, is charged to perform wash trading. The largest sale for tokens in that collection was only 3.95 ETH [68], before Jan 12th, 2022. However, one seller sold the #13824 token of Meebits for \$49.5 million [69] this day to one buyer, which is ten million times greater than the largest sale before this day. Moreover, the token was again sold back from the previous buyer to the previous seller at nearly the same price. This aligns with the definition of wash trading, where a small group artificially generates a significant trading volume for an NFT project within a short period. The huge amount of money between these two trades indicates that this is wash trading. Note that not all trading behaviors are wash trading. While the value of NFTs is often determined by market speculation and sentiment, this does not inherently categorize all trades as wash trades. According to the definition, wash trading is a specific practice where a trader buys and sells an asset with the same entity on both sides of the transaction frequently to create misleading market activity.

Identifying existing incidents. Previous studies [9,70–72] have proposed several methods to detect wash trading. On the one hand, some of them adopt heuristic rule-based detection for NFT trades. For example, von Wachter et al. [72] examined the top 52 NFT collections with the greatest impact on the secondary market and identified six different wash trading patterns, along with creating rules to detect them. On the other hand, some have proposed formal methodologies to find wash trading to reduce false negatives to some extent. For instance, Bonifazi

et al. [70] built a mathematical model for NFTs by three defined parameters to detect wash trading. However, despite numerous studies exist, none of them have specifically focused on the characteristics of attackers themselves, e.g., how many scam campaigns that perform wash trading are there in the wild.

Best practice. It is necessary to defend against wash trading to eliminate fake prosperity in the NFT ecosystem. One possible approach is to promptly inform the community about projects involved in wash trading in real time, with the goal of quickly alerting investors. The challenge lies in identifying wash trading shortly after it begins. Researchers can adopt heuristics for detection, which can be assisted by insights from prior instances of wash trading. Further, some advanced techniques like artificial intelligence-assisted detection can be explored.

5.2. Arbitrage

Considering financial property, arbitrage exists within the NFT ecosystem. Specifically, two types are observed, i.e., collection offer arbitrage and flashloan-based arbitrage.

Sub-issue I: Collection offer arbitrage. A collection offer is like a wanted sign for NFTs [73], which is a feature offered by several trading markets, like Opensea, X2Y2, and LooksRare [74,75]. Specifically, NFT holders can sell an NFT to another account that has published a collection offer and obtained the attached money. Thus, if a bot constantly monitors collection offers raised in NFT markets and NFT tradings, once an NFT is sold and an existing collection offer offers a higher value for that NFT, the bot will automatically buy it and immediately sell it to the collection offer. For example, the transaction 0x4b5e³ is a collection offer arbitrage conducted by 0x9e93⁴. Specifically, 0x9e93 first bought the token from 0x4d9⁵ by \$318.98 and then again sold it to 0x7d2d⁶ by \$456.04 in a single transaction on Opensea, which is in line with the format of collection offer arbitrage.

Sub-issue II: Flashloan-based arbitrage. Flashloan is a type of feature provided by some lending DApps, like Uniswap [76]. By paying a relatively small fee, flashloan allows one account to borrow many tokens, which have to be returned within a single transaction. Therefore, for individual investors, to execute flashloan-based arbitrage, they first borrow funds, then make a series of trades to profit from the price difference, and finally repay the flash loan with a small fee and return the borrowed assets. Some secondary marketplaces allow bidding on NFTs (see Section 2.3). Therefore, it is possible to bid one token at prices higher than the current market rates. As a result, some arbitrageurs seize the opportunity to profit from the price difference between the bid of a buyer and the listed token, engaging in profitable arbitrage. For example, Ape Coin recently engaged in flashloan-based arbitrage [77]. Here is how it went down: first, the arbitrageur obtained a flashloan from a lending platform; then, using the borrowed funds, the arbitrageur purchased an Ape Coin; finally, the token was sold on LooksRare to the highest bidder, fetching a price six times higher than the original purchase. To ensure that this transaction would be the first one in the next block, the arbitrageur transferred 38 ETH to F2Pool, a collaborative mining pool, encouraging block discovery and prioritizing transaction confirmation. Despite this significant cost, the arbitrageur still gained a substantial profit of 20 ETH in the end.

Identifying existing incidents. While there are published papers that address collection offer arbitrage, methods for identifying flashloan-based arbitrage remain less explored. First, for identifying existing incidents of collection offer arbitrage, researchers can focus on the key step, i.e., an account purchases and sells NFTs within a single transaction [78]. If the price of purchasing is lower than the price of selling, we assume that it is a collection offer arbitrage. The main difference between

NFTs and regular flashloan-based arbitrage is the presence of NFTs. Researchers can use a straightforward method to identify these incidents. Specifically, we can monitor flashloan activities on lending platforms to see if borrowers are using these flashloans for trading, which can help pinpoint potential arbitrage scenarios.

Best practice. Note that, compared to other security issues, arbitrage is a legitimate practice of profiting from market rules. Therefore, its malicious intent is relatively minor. One strategy to defend against such issues could involve front running. Specifically, defenders can identify the collection offering arbitrage opportunities before the arbitrageur does and swiftly execute the trade. Subsequently, they may return assets, ensuring that the arbitrageur does not implement any profit. To make this strategy effective, defenders need to swiftly identify opportunities and promptly execute transactions on the blockchain. The research community has delved into how to make flashloan-based arbitrage bots faster than others do [79].

5.3. Rug pull

A rug pull is a set of behaviors in which scammers launch a project, attract investors' money, and then suddenly shut down the project or ghost investors while taking all invested assets. AniMoon [80] is a typical example. First, the rug puller launched the project and advertised it through their Twitter and official websites [81]. Second, after attracting enough investors and accumulating enough funds, the rug puller withdrew over \$6.3 million worth of ETH from the project's smart contract [82]—just 11 days after its launch. Third, they abandoned the project permanently by shutting down the Twitter account and stopping the update of its official website. Now, the floor price of this project is low (less than 0.01 ETH) [83], which indicates the unpopularity caused by the rug pull. Note that NFT-related DApps may also conduct a rug pull. For example, SudoRare, a fork NFT exchange between two famous secondary markets, was reported as a rug pull [84]. Six hours after the launch of the platform, the team rugged the pull, stole the tokens from users, and withdrew over \$820,000 worth of digital tokens⁷.

Identifying existing incidents. There are studies on NFT scams and effective methods for detecting rug pulls already exist. Huang et al. [85] effectively used rules-based approaches to identify instances of rug pulls that have already occurred. Their method involved detecting specific patterns indicative of such scams, particularly rapid fund withdrawals following a project's launch. NFT projects can be categorized based on their presence on social media, and monitoring these platforms can be crucial. For projects without active social media engagement, employing heuristic observations can help in identifying potential rug pulls from various angles, such as extended inactivity on developers' social media accounts.

Best practice. While the lack of intrinsic value in NFTs can increase their vulnerability to rug pulls, this does not make preventing them impossible. Rug pulls usually happen because of malicious intent or poorly executed projects, not just because NFTs lack intrinsic value. Therefore, it is possible to prevent rug pulls by focusing on security measures and ensuring better project design and market transparency. Quickly assessing if a project is going to commit a rug pull in its early stage is crucial for defense against rug pulls. Therefore, it requires effectively using limited and early-stage data from NFT projects. In general, projects that tend to be rug pulled exhibit the following two characteristics in their early stages. First, there is a substantial minting of tokens to earn mint fees. Second, they may display excessively high prices in the market. Therefore, detection can be devised by focusing on these two aspects.

5.4. Copyright theft

Each NFT is linked to a piece of digital content. Thus, scammers may steal copyright from famous digital assets, e.g., images, without

³ <https://etherscan.io/tx/0x4b5ea5...>

⁴ <https://etherscan.io/address/0x9E9346...>

⁵ <https://etherscan.io/address/0x4d9183...>

⁶ <https://etherscan.io/address/0x7d2d61...>

⁷ <https://app.zerion.io/0xbb42f7....>

permission, cheating the public and gaining illegal profit. We divide this into two categories, i.e., sleepmint and counterfeiting NFTs.

5.4.1. Sleepmint

Sleepmint refers to a kind of behavior, where the scammer exploits a backdoor in the contract, pretending to purchase currency from a famous artist and subsequently selling it to the victim at a higher price. Specifically, first, the attacker creates a smart contract with a hidden backdoor, allowing them to transfer the token from the owner to themselves without the owner's approval. Then the attacker initially mints a token associated with content to a celebrity and transfers it back from that celebrity via a backdoor. Since the blockchain records that the celebrity previously owned the token, the attacker then reclaims ownership. Finally, the attacker claims that the NFT project was created by the celebrity and they purchased the token from the celebrity. As a result, the attacker can sell it at a high price to a victim. Bleeple, a famous artist active in the blockchain community, was once sleepminted [86]. He was targeted by an anonymous person *Personne*, who minted an NFT⁸ (the token ID is 40914) to Bleeple, as the second edition of one Bleeple's famous digital content, i.e., *Everyday: the first 5,000 days*. After that, he drew it back and claimed that the token was created by Bleeple. Finally, he put it on sale on OpenSea [30] and Rarible [87] for sale. Fortunately, these two platforms both eventually deactivated the token and forbade the sale.

Identifying existing incidents. According to the pattern discussed above, the key to identifying a sleepmint is to detect, for a token, whether there exists a transfer event conducted by an unapproved account after a mint event. We note that Guidi and Michienzi [88] once investigated this issue, although their definition of sleepmint was overly broad that may lead to several false positives, which required some manual effort to eliminate them.

Best practice. Guidi and Michienzi [89] proposed a method for preventing sleepmint by probing the actions of attackers, which were extracted from their sleepmint detection results. Specifically, they leveraged existing incidents. They examined whether a transaction showed signs of connecting with an address linked to previous attackers involved in sleepminting. If such a connection was identified, the transaction would be blocked.

5.4.2. Counterfeiting NFT

Counterfeiting tokens in blockchain platforms is a widely spread malicious behavior. In the case of fungible tokens, counterfeiting can generally be conducted by imitating token names [90]. When it comes to NFTs, except for name counterfeiting, back-end digital assets can also be the target.

Sub-issue I: Name counterfeiting. This type of counterfeit NFT usually has an identical or confusingly similar name to the victim NFT, exploiting the feature that blockchain does not place any name restrictions. For example, BAYC [3] has an identical fake token⁹ that has been flagged by Etherscan. Fake NFT Moonbirds¹⁰ have a similar name to the official NFT Moonbirds [91].

Sub-issue II: Image counterfeiting. Image counterfeiting is the illicit act of stealing images from their original sources to generate NFTs. This practice commonly targets art images created by artists and other officially sanctioned NFT images. Using these artworks without authorization constitutes another form of creating counterfeit NFTs. A typical example is the *Starry Night Dog* [92]. The *Starry Nights Dog* is the most famous painting series of artist Aja Trier, which was turned into an NFT without the authorization of the artist.

Identifying existing incidents. Das et al. [9] measured the existence of counterfeit NFTs in their paper, used the Levenshtein distance to mea-

sure the similarity of NFT names, and employed perceptual hashing algorithms to assess the similarity of NFT images. However, it focused only on the NFTs that have long names (i.e., more than seven chars). Therefore, they left some NFT collections with short names undetected, which introduced false negatives. For image comparison, only the similarity between images of different NFT collections was considered, disregarding comparisons with existing art images. Additionally, the perceptual hashing algorithm itself has a relatively slow execution speed.

Best practice. For investors, to avoid being deceived by counterfeit NFTs, precautions should start with contract creation. Real-time monitoring of NFT contract creation allows for the identification of their NFT contract names. By comparing these names with official NFT names, potential name forgery can be detected. Identifying image forgery involves recognizing the URI where NFT assets are stored. Typically, this can be accomplished by calling the contract's `tokenURI` to retrieve metadata URIs, from which image URIs can be read. These URIs can be used to fetch images for similarity comparison. Establishing a database comprising existing NFT images and artwork images facilitates similarity comparisons for newly created NFT images. Wang et al. [93] introduced a self-defense copyright protection method for NFT image art by embedding specific information within NFT images, enabling the detection of potential NFT image theft.

6. Auxiliary service layer

This section focuses on the security issues located in the auxiliary service layer, like the security issues related to front-end of secondary marketplaces. Stöger et al. [10] emphasized the risks of NFT hijacking that arised from interactions with auxiliary components, highlighting the importance of security in the auxiliary layer.

6.1. Website exploitation

NFT-related DApps, while leveraging traditional web pages to offer user-friendly interfaces, inherit the risks and vulnerabilities of conventional Web2 environments, including threats like malicious script upload, Application Programming Interface (API) exploitation, User Interface (UI)-based fraud, and Distributed DoS (DDoS) attack.

Sub-issue I: Malicious script upload. Attackers exploit vulnerabilities on the website to upload malicious scripts. These scripts are able to alter NFT information (like URIs), redirect users to fraudulent sites, or even manipulate transactions. A prime example is PREMINT [94], an NFT tool that helps NFT projects create access control conditions for NFT mints to prevent bots from minting the NFT supply. It provides a front-end visual interface to help users visualize the NFT mint settings. Primint front-end integrates boomerang [95], which allows users to upload images to the back-end server. However, due to a vulnerability within the boomerang, attackers can bypass the verification rules and upload a malicious JavaScript file [96]. Using malicious code, attackers can generate deceptive transactions. If victims unknowingly sign these, it gives the attackers the approval to transfer the victims' NFTs and Ether [97,98].

Sub-issue II: API exploitation. The API on NFT secondary marketplaces may be exploited to get sensitive information, such as personal information and the price of a specific NFT. The API of the project made by Apes can leak user information, such as email addresses and social media profiles [99]. Coincidentally, OpenSea NFT Marketplace once suffered this one, with the exploiter withdrawing approximately 332 ETH [100–102] worth in 2022. Take the biggest sale BAYC as an example¹¹. First, to avoid huge canceling fees, the victim managed to transfer his or her NFTs to another wallet. This action should have canceled the listings of NFTs, and these NFTs should not appear at the front-end of OpenSea [30]; however, they were still listed on Rarible [87], which

⁸ <https://etherscan.io/token/0x5FBbAC...>

⁹ <https://etherscan.io/address/0x28ed3a...>

¹⁰ <https://etherscan.io/address/0x335cbe...>

¹¹ <https://etherscan.io/token/0xbc4ca0...>

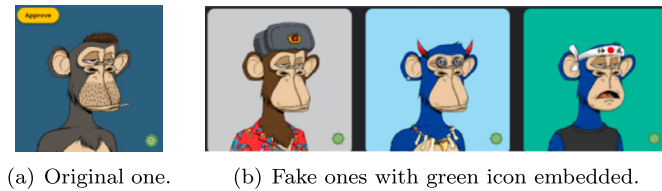


Fig. 2. Real Bored Ape Yacht Club (BAYC) vs. fake BAYC.

gained the data through OpenSea's 'Get All Listings' API [103]. The victim thought that the listings were canceled and transferred the NFTs back to the original wallet, which set the price as 0. The opportunist rapidly bought the tokens from the former listings without any price and made a huge loss to the victim for approximately 332 ETH.

Sub-issue III: User interface-based fraud. The design of the UI could provide the possibility for the scam of victims. It has been reported that attackers exploited the user interface of markets to commit fraud [104, 105]. Typically, on Swap.Kiwi, an NFT secondary marketplace, an official and verified NFT would be labeled a green checkmark by the market, as shown in Fig. 2(a). However, a malicious user created a fake BAYC NFT with a green checkmark overlapping manually, as shown in Fig. 2(b). The victim did not notice such a subtle difference. Therefore, he swapped his real BAYC NFT with the fake one on Swap.Kiwi. This caused nearly \$570,000 in economic losses to the victims.

Sub-issue IV: Distributed DoS attack. A Distributed DoS (DDoS) attack involves overwhelming a website or online service with a flood of internet traffic, causing disruptions or making the site inaccessible, which can also impact NFT platforms [106]. The NFT market LooksRare [107] once suffered a DDoS attack, preventing users from connecting their wallets to trade NFTs [108]. Similarly, the Axie Infinity trading platform has also experienced such attacks [109].

Identifying existing incidents. Targeting these traditional Web2 vulnerabilities, there are some powerful ready-made scanners like Nessus [110] and Burp Suite [111], which can detect potential web vulnerabilities.

Best practice. To prevent such vulnerabilities, we summarize best practices from different perspectives. Standing from the developer side, they should conduct regular code reviews and static analysis, adhere to secure coding standards, and implement stringent input validation. On the user side, they should verify the authenticity of the visited websites and avoid clicking on any links or downloading attachments from unknown sources.

6.2. NFT-themed phishing

Phishing is a deceptive practice where attackers pretend themselves as trustworthy entities to extract sensitive information from victims. NFT-themed phishing includes website phishing, phishing trojans disguised as NFT games, spam NFTs that use NFTs as phishing link carriers, and hacking NFT project social media to spread phishing links. The following provides a detailed overview.

6.2.1. Website phishing

In website phishing, attackers use fraudulent websites that mimic the appearance and domain names of official sites to trick users into signing malicious transactions, thereby stealing their assets. Website phishing is also practical in the NFT ecosystem. The typical case is <https://moonbirds-exclusive.com/>, which targets the famous project Moonbirds [112]. Upon entering the website, it requests users to connect their wallets. When the user clicks the connect wallet button, a signature request warning pops out on MetaMask. This is the first malicious transaction, requesting users to sign using `eth_sign`, which is the highest authority signature [113]. Coupled with the address of the signer, these signature data can be reused to sign any other transaction. The second malicious transaction is `SetApproveForAll` authorization, and

the authorized address can freely transfer the NFTs of the grantor [114]. Attackers can steal the signatures of users and tokens through phishing.

Identifying existing incidents. DNStwist [115] generates similar domains by squatting from an official domain to find potential phishing websites.

Best practice. The best practice to defend against website phishing attacks is to constantly monitor the creation of web pages and set alerts for any suspicious activities. He et al. [116] and Roy et al. [117] set up a cert-stream server to receive live updates from the certificate transparency log network, thus retrieving suspicious domains, including domains generated by squatting in real time.

6.2.2. Phishing trojans

Phishing trojans are designed as ordinary software, appearing genuine but actually containing malicious programs that steal crucial user information [118]. NFT-themed phishing trojans have emerged within the ecosystem. In the CthulhuWorldP2E NFT game project, the operator distributed malware masked as game beta files on their discord channel [119]. This malicious software was later identified as RedLine Stealer. Note that RedLine Stealer [120], available since 2020 [121], is frequently concealed within Phishing trojans. Its latest iteration is capable of extracting data from cryptocurrency wallets, especially those functioning as browser extensions, such as MetaMask.

Identifying existing incidents. Paying close attention to user reports and complaints regarding unusual software behavior or unauthorized access to information.

Best practice. The best practice for combating phishing trojans should be approached from a user perspective, aiming to enhance user awareness and safety.

6.2.3. Spam NFTs

Using a unique airdrop mechanism, NFTs containing phishing links in their description are mass distributed to users, similar to sending spam emails [122]. The description of the NFT contains language that lures users to click on phishing links. The NFT collection 'Lets Walk' is typical spam NFT phishing [123]. First, the tokens of the collections are airdropped to different users without their approval. Second, there is a phishing link in the description of this NFT, which is a vehicle to spread phishing sites. These NFT descriptions contain enticing language to lure users to a phishing site for free mint. Consequently, the phishing site lures the victim to sign a malicious transaction, which enables them to transfer all their digital assets.

Identifying existing incidents. Forta [124] provides a heuristic rule-based approach [125] for detecting spam NFTs, considering factors such as false total supply and the scale of an airdrop.

Best practice. After understanding this security issue, we summarize the best practices from various perspectives. Specifically, from the standpoint of users, they need to be careful about any sudden appearance of unfamiliar links in their accounts. If so, they should not be deceived by them. From the platform perspective, educating users about this situation and standardizing the descriptions of coins on their platform can be beneficial. From the viewpoint of researchers, deploying a monitoring system to oversee the descriptions of airdropped tokens is malicious.

6.2.4. Hacking social media

Recall that some NFT collections constantly attract investors through social media [126], such as Twitter [127], Discord [28], and Telegram [128] (see Section 2.3). Attackers could hijack these social media to post fake news or distribute phishing links. The most renowned NFT project, BAYC, had its discord channel hacked [3], where hackers posted phishing links, stealing approximately \$360,000 worth of NFTs [129,130].

Identifying existing incidents. Whether NFT social media has been hijacked can ultimately be confirmed only by the account owner; hence, it can be assessed only based on the official disclosure of information. Typically, after a social media hack, attackers tend to post numerous false

messages that are often related to giveaways, including phishing links to lure users into clicking. By continuously monitoring official project social media channels, the discovery of phishing links suggests potential account hijacking.

Best practice. The best way for users to prevent this sub-issue is to use strong and unique passwords, or multi-factor authentication. Moreover, they should be cautious with suspicious emails and links.

6.3. Asset-related issue

Recall that assets are stored in different storage services and are linked via URIs. However, the combination of on-chain data and off-chain assets raises some security concerns that could harm this connection. Specifically, two kinds of security incidents are observed, i.e., asset inaccessibility and metadata tampering.

Sub-issue I: Asset inaccessibility. The value of each NFT solely depends on its bound real-world digital asset, where the indexing is accomplished by URIs. In other words, if a URI is invalidated, making the real-world asset inaccessible, the corresponding NFT will be valueless.

Sub-issue II: Metadata tampering. The metadata of a token contain the corresponding URI to the bound asset. The ability to change NFT metadata is controlled by the functions written into the smart contract. In some cases, the contract may intentionally allow parties other than the NFT owner to modify metadata. This could be for various reasons, such as enabling updates or adjustments by developers or platforms or simply the malicious intentions. Therefore, others except the owner may have the chance to change the metadata. Consequently, if the metadata are altered maliciously, the token loses its meaning. The ERC-721 standard for NFTs actually permits the modification of a token's metadata.

Identifying existing incidents. For the first sub-issue, the current work [131] tried to check the data accessibility from three stages, i.e., the tokenURI to metadata, metadata to AssetURI, and AssetURI to Asset. Consequently, according to the statistics, 10 centralized platforms accounted for hosting 79.04% of NFTs' off-chain assets, indicating that NFTs were stored in a centralized way to some extent. However, among them, only 33.77% of NFTs' assets were retrieved. This result suggested that approximately two-thirds of NFT have already lost their value because of the asset inaccessibility. For the second sub-issue, Das et al. [9] once monitored the metadata of all external assets on OpenSea three times over a period of six months at regular intervals. They observed that 2.89% of metadata and 1.15% of assets had changed between the first two and the last two crawls, respectively. Note that this method assumes that all changes are malicious by default, which is a limitation. However, there are cases where metadata changes are necessary because of asset inaccessibility. Therefore, researchers should conduct further checks to determine if the original assets remain accessible to confirm whether such actions are indeed malicious.

Best practice. Two approaches could be adopted to mitigate asset inaccessibility. First, researchers could tackle the problem of assets being inaccessible through storage services. However, this is challenging because it requires us to modify the storage service protocol to ensure perpetual asset accessibility. Second, researchers could concentrate on the URIs of NFTs. The NFT standard permits the modification of the URI of an NFT. However, this also serves as a significant vulnerability since it is unclear whether the changes to NFTs are malicious, as elaborated in the next incident. Moreover, at present, there are no widely recognized suggested defense methods for metadata tampering. However, some alternative approaches exist. One of them is to monitor the permissions for the relevant functions that are able to change the metadata in the contract, ensuring that only the owner or authorized entities can invoke it. Another strategy involves implementing a dynamic monitoring system. This system checks whether the metadata of tokens in the market have been modified and whether their original assets are still accessible.

6.4. Private key compromised

Recall that a crypto wallet is a digital tool that manages blockchain-based assets (see Section 2.1). Users use cryptographic public-private key pairs to access wallets and sign transactions. However, their private key or digital signature might be leaked, e.g., through a phishing site, which may result in loss of account funds. LiveArtX [132], an NFT art platform, serves as a typical example. The compromise of its wallet private key led to unauthorized NFT transfers and incurred a loss of approximately \$39,000.

Identifying existing incidents. After a private key is compromised, it can result in an abnormal decrease in account funds, often transferred to previously unassociated accounts.

Best practice. The hardware wallets are used as physical vessels to store private keys, securely enclosing private keys and preventing online exposure. The use of a multi-signature wallet mandates the presence of multiple keys for transaction approval to increase security. Elevating phishing awareness enlightens users about the perils of inadvertently divulging keys to deceitful schemes. By integrating these strategies, the potential risks linked to private key breaches can be significantly diminished.

7. Discussion

7.1. Lessons learned

Gap between industry and academy. Our systematic survey shows that there is a lack of comprehensive understanding of NFT security issues in academic research. Current research on NFT security addresses only a limited portion of overall NFT security issues. In particular, for all security issues, the majority have not been discussed in published papers on NFT security. Therefore, a comprehensive understanding of NFT security issues in academic research is lacking, indicating a notable gap between academic and industry awareness in this field.

Detectable security issues. Many security issues have not been extensively measured on a large scale in previously published articles. Note that some security issues are isolated and not widespread, e.g., UI-based fraud (see Section 6.1), making it challenging to measure them comprehensively on a large scale. However, we identify security issues that are common and can be systematically identified or measured. Among these, academic research has explored issues like wash trading and asset inaccessibility. Nevertheless, many security issues, such as spam NFTs, have not been extensively measured on a large scale in previously published articles. These security issues that can be detected or measured carry significant academic value. In this paper, we also offer initial detection strategies to inspire future research in this area.

Limited understanding of attackers. Even if certain security issues have been thoroughly studied, our understanding of attackers is still limited. When exploring NFT security issues in depth, despite an understanding of how to detect them and how common they are, most studies do not thoroughly study the attackers themselves. Take wash trading as an example. Many papers have studied this problem, but none have focused on understanding attackers. Consequently, our knowledge of attackers is limited, and we still have unanswered questions, like whether attackers work together in organized groups.

Absence of a defense system. Apart from a few specific security issues, we have not seen researchers suggesting defense systems for most security problems. Considering the widespreadness, severity, and intractability of NFT security issues, having a defense system is of paramount importance in addressing these security concerns. While it may not be possible to entirely eliminate some security issues, we can still implement methods, like providing valuable recommendations by automatic processes, to reduce the risks associated with these problems. However, except for a few specific security issues, such as sleepmint (even though we have identified its limitations, as discussed in Section 5.4.1), we have

not observed any researchers proposing defense systems for most security issues.

7.2. Threats to validity

Some limitations exist in our work that may affect its validity. First, we gather reports from renowned sources of technical reports and peer-reviewed papers. Consequently, we do not consider reports from smaller or emerging companies and communities or all pre-print websites. We argue that we have made our best effort to collect data from the most dependable and available sources. Although our data collection procedure is heavily based on a manual process, to establish the most reliable dataset, each report or paper is meticulously reviewed by two students who have paid attention to blockchain for more than two years. Second, we do not discuss the detection or defense methods for all security issues, as some of them are specific cases and are not applicable in a general context. Nevertheless, we either summarize existing research or outline potential methods for addressing all general security issues.

8. Related work

While surveys and SoKs related to NFTs exist, studies specifically addressing security aspects are lacking. This gap in the literature underscores the need for focused research on the security vulnerabilities associated with NFTs. In 2021, Wang et al. [133] provided the first systematic study of the NFT ecosystem, offering a comprehensive overview of NFT solutions, technical components, protocols, standards, and their security evolution, along with discussions on design models, opportunities, and challenges. In 2022, Zhou et al. [46] conducted another comprehensive overview, with a focus on DeFi incidents and security. They analyzed 77 academic papers, 30 audit reports, and 181 real-world incidents [46]. In 2019, Praitheshan et al. [134] conducted a survey on DeFi in the Ethereum network, where they identified 16 security concerns and provided some solutions. Saad et al. [135] categorized 22 vulnerability factors into three areas—blockchain structure, Peer-to-Peer (P2P) system, and blockchain applications—and studied their relationships among miners, mining pools, users, and exchanges. In 2020, Chen et al. [136] found 40 vulnerabilities, examined 29 real-world attacks, and discussed 51 defenses related to blockchain technology. In 2021, Werner et al. [137] conducted a comprehensive overview of DeFi, covering its basic and security issues. Our research is centered on NFTs, which represent a specific aspect of DeFi. This lack of research has motivated our work, which is the first and aims to contribute to future research in this area.

9. Conclusion

NFT is a key component of the Web3 ecosystem and has attracted great interest from both academia and industry. In this paper, we present the first SoK of NFT security, covering 12 major categories of NFT security issues, by scrutinizing 176 security incidents. We provide potential detection and defense methods for those unsolved security issues and show that NFT security issues are widespread, severe, and intractable. We identify the gap between industry and academia, which will boost future research on NFT security.

CRedit authorship contribution statement

Kai Ma: Investigation, Methodology, Writing – original draft. **Jintao Huang:** Data curation, Methodology, Writing – original draft. **Ningyu He:** Investigation, Project administration, Validation. **Zhuo Wang:** Validation, Visualization. **Haoyu Wang:** Conceptualization, Supervision, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This work was partly supported by the Knowledge Innovation Program of Wuhan-Basic Research, Key R&D Program of Hubei Province (Nos. 2023BAB017 and 2023BAB079), and HUSTCSE-Hongxin Joint Research Center for Network Security.

References

- [1] The year of the nft: How an emerging medium went mainstream in 2021, <https://www.artnews.com/list/art-news/artists/2021-year-of-the-nft-1234614022/>, 2021.
- [2] Token uris in nft projects, <https://knowledge.pinata.cloud/en/articles/6461213-token-uris-in-nft-projects>, 2023.
- [3] The bored ape yacht club official website, <https://boredapeyachtclub.com/>, 2022.
- [4] Bored ape yacht club, <https://opensea.io/collection/boredapeyachtclub/analytics>, 2023.
- [5] Hack on bored ape yacht club nfts leads to \$3m simian oblivion, <https://www.theguardian.com/technology/2022/apr/26/bored-ape-yacht-club-nft-hack-theft-art-simian-oblivion>, 2022.
- [6] Hacker steals \$1.4 million in nfts from collector in one sweep, <https://www.vice.com/en/article/pkp7pm/hacker-steals-dollar14-million-in-nfts-from-collector-in-one-sweep/>, 2022.
- [7] Hackers stole Ethereum worth \$185,000 ethereum from Bill Murray after NFT charity auction, <https://www.marketwatch.com/story/hacker-steals-185-000-in-ethereum-from-bill-murray-after-nft-charity-auction-11662319617>, 2022.
- [8] Worldwide nft heists tracker, <https://www.comparitech.com/blog/vpn-privacy/nft-heists/>, 2023.
- [9] D. Das, P. Bose, N. Ruaro, et al., Understanding security issues in the NFT ecosystem, arXiv:2111.08893, 2022, preprint, <https://arxiv.org/abs/2111.08893v3>.
- [10] F. Stöger, A. Zhou, H. Duan, et al., Demystifying Web3 centralization: the case of off-chain NFT hijacking, in: Financial Cryptography and Data Security, Springer Nature, Switzerland, 2023, pp. 82–199, https://doi.org/10.1007/978-3-031-47751-5_11.
- [11] What is a block in the crypto blockchain, and how does it work?, <https://www.investopedia.com/terms/b/block-bitcoin-block.asp>, 2022.
- [12] Ethereum accounts, <https://ethereum.org/en/developers/docs/accounts/>, 2022.
- [13] How blocks are added to a blockchain, explained simply, <https://www.coindesk.com/learn/how-blocks-are-added-to-a-blockchain-explained-simply/>, 2023.
- [14] What are consensus protocols?, <https://decrypt.co/resources/consensus-protocols-what-are-they-guide-how-to-explainer>, 2019.
- [15] What is proof of work (pow) in blockchain?, <https://www.investopedia.com/terms/p/proof-work.asp>, 2023.
- [16] What is proof of stake?, <https://www.mckinsey.com/featured-insights/mckinsey-explainers/what-is-proof-of-stake>, 2023.
- [17] Bitcoin official website, <https://bitcoin.org/>, 2023.
- [18] Introduction to smart contracts, <https://ethereum.org/en/developers/docs/smart-contracts/>, 2023.
- [19] Erc-721 token standard, <https://ethereum.org/en/developers/docs/standards/tokens/erc-721/>, 2023.
- [20] Eip-1155: Multi token standard, <https://eips.ethereum.org/EIPS/eip-1155>, 2022.
- [21] Solana nfts, <https://solana.com/developers/nfts>, 2023.
- [22] Solana official, <https://solana.com/>, 2023.
- [23] Bep-721, <https://academy.binance.com/en/glossary/bep-721>, 2023.
- [24] Binance official, <https://www.binance.com/>, 2023.
- [25] A brief history of defi, <https://decrypt.co/resources/a-brief-history-of-defi-learn>, 2023.
- [26] Mobile wallet vs. extension wallet: Which one should you opt for?, <https://medium.com/onto/mobile-wallet-vs-extension-wallet-which-one-should-you-opt-for-51db6eab088b>, 2023.
- [27] Twitter, <https://twitter.com/>, 2023.
- [28] Discord, <https://discord.com>, 2023.
- [29] Instagram, <https://www.instagram.com/>, 2023.
- [30] Opensea official, <https://opensea.io/>, 2021.
- [31] Looksrare official, <https://looksrare.org/>, 2022.
- [32] X2y2 official, <https://x2y2.io/>, 2021.
- [33] How to sell an nft: a comprehensive guide, <https://hedera.com/learning/nfts/how-to-sell-an-nft>, 2022.
- [34] What is an nft collection?, <https://www.linkedin.com/pulse/what-nft-collection-gem-invest-group>, 2023.
- [35] What is a verified account or badged collection?, <https://support.opensea.io/>, 2023.

- [36] Wyvern protocol, <https://wyvernprotocol.com/>, 2019.
- [37] Introducing seaport protocol, <https://opensea.io/blog/announcements/introducing-seaport-protocol/>, 2022.
- [38] How do nft marketplaces make money?, <https://nftclub.com/how-do-nft-market-places-make-money/>, 2022.
- [39] What are nft royalties and how to earn from them?, <https://www.blockchainx.tech/nft-royalties-how-they-work>, 2022.
- [40] Beosin official website, <https://beosin.com/>, 2023.
- [41] Slowmist official website, <https://www.slowmist.com/>, 2023.
- [42] Blocksec official website, <https://blocksec.com/>, 2023.
- [43] Certik official website, <https://www.certik.com/>, 2023.
- [44] Defihacklabs, <https://github.com/SunWeb3Sec/DeFiHackLabs>, 2023.
- [45] Defi rekt database, <https://de.fi/rekt-database>, 2023.
- [46] L. Zhou, X. Xiong, J. Ernstberger, et al., SoK: decentralized finance (DeFi) attacks, in: Proceedings of the 2023 IEEE Symposium on Security and Privacy (SP), IEEE, 2023, pp. 2444–2461, <https://doi.org/10.1109/SP46215.2023.10179435>.
- [47] When “safemint” becomes unsafe: Lessons from the hypebears security incident, <https://blocksecteam.medium.com/when-safemint-becomes-unsafe-lessons-from-the-hypebears-security-incident-2965209bda2a>, 2022.
- [48] Security event case study — omni (nft money market) hit by re-entrancy attack, <https://degis.medium.com/security-event-case-study-omni-nft-money-market-hit-by-re-entrancy-attack-ef65fce5003c>, 2022.
- [49] Slowmist: An analysis of the attack on the omnix nft platform, <https://slowmist.medium.com/slowmist-an-analysis-of-the-attack-on-the-omnix-nft-platform-51f89a6042ca>, 2022.
- [50] Blocksecteam’s analysis of omni protocol, <https://twitter.com/BlockSecTeam/status/1546141457933025280>, 2023.
- [51] Checks effects interactions, https://fravoll.github.io/solidity-patterns/checks_effects_interactions.html, 2023.
- [52] How akutar nft loses 34m usd, <https://blocksecteam.medium.com/how-akutar-nft-loses-34m-usd-60d6cb053dff>, 2022.
- [53] Dos with failed call, <https://swcregistry.io/docs/SWC-113/>, 2023.
- [54] A. Ghaleb, J. Rubin, K. Pattabiraman, et al., detecting gas-related vulnerabilities in smart contracts, in: Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, 2022, pp. 728–739.
- [55] N. Grech, M. Kong, A. Jurisevic, et al., MadMax: surviving out-of-gas conditions in Ethereum smart contracts, Proc. ACM Program. Lang. 2 (OOPSLA) (2018) 1–27, <https://doi.org/10.1145/3276486>.
- [56] Z. Liao, S. Hao, Y. Nan, et al., SmartState: detecting state-reverting vulnerabilities in smart contracts via fine-grained state-dependency analysis, in: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, ACM, 2023, pp. 980–991, <https://doi.org/10.1145/3597926.3598111>.
- [57] Access control openzeppelin docs, <https://docs.openzeppelin.com/contracts/3.x/access-control>, 2023.
- [58] S. Yang, J. Chen, Z. Zheng, Definition and detection of defects in NFT smart contracts, in: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, ACM, 2023, pp. 373–384, <https://doi.org/10.1145/3597926.3598063>.
- [59] A. Ghaleb, J. Rubin, K. Pattabiraman, AChecker: statically detecting smart contract access control vulnerabilities, in: Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), IEEE, 2023, pp. 945–956, <https://doi.org/10.1109/ICSE48619.2023.00087>.
- [60] The sandbox official website, <https://www.sandbox.game/en/>, 2022.
- [61] Style guide, <https://docs.soliditylang.org/en/latest/style-guide.html#local-and-state-variable-names>, 2023.
- [62] Unlimited minting of bored ape yacht club nfts?, https://www.reddit.com/r/CryptoCurrency/comments/sn8x78/unlimited_minting_of_bored_ape_yacht_club_nfts/, 2022.
- [63] \$34m locked due to contract vulnerabilities in akutar: Beosin’s full analysis, <https://beosin.com/resources/34m-locked-due-to-contract-vulnerabilities-in-akutar?lang=en>, 2022.
- [64] Breaking down v1 cryptopunks: The first cryptopunks release, <https://www.startwithnfts.com/posts/breaking-down-v1-cryptopunks-the-first-cryptopunks-release/>, 2022.
- [65] P. Qian, J. He, L. Lu, et al., Demystifying random number in ethereum smart contract: taxonomy, vulnerability identification, and attack detection, IEEE Trans. Softw. Eng. 49 (7) (2023) 3793–3810, <https://doi.org/10.1109/TSE.2023.3271417>.
- [66] Meebits accident, <https://de.fi/rekt-database/meebits>, 2023.
- [67] Wash trade, Wikipedia, https://en.wikipedia.org/wiki/Wash_trade, 2022.
- [68] A meebit nft sold for \$49.5m in ethereum—but there’s a catch, <https://cryptobriefing.com/meebit-nft-sold-for-49-5m-ethereum-with-catch/>, 2022.
- [69] Meebits #13824 transaction, <https://etherscan.io/tx/0xcd812ee7b10a486201ff8e79d417d9ee687cb80733bb35ccf2716e846d6e47d8>, 2022.
- [70] G. Bonifazi, F. Cauteruccio, E. Corradini, et al., Performing wash trading on NFTs: is the game worth the candle?, Big Data Cogn. Comput. 7 (1) (2023) 38, <https://doi.org/10.3390/bdcc7010038>.
- [71] X. Wen, Y. Wang, X. Yue, et al., NFTDisk: visual detection of wash trading in nft markets, in: Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems, 2023, pp. 1–15, <https://doi.org/10.1145/3544548.3581466>.
- [72] V. von Wachter, J.R. Jensen, F. Regner, et al., NFT wash trading: quantifying suspicious behaviour in NFT markets, in: Financial Cryptography and Data Security. FC 2022 International Workshops, Springer International Publishing, pp. 99–311, 2023, https://doi.org/10.1007/978-3-031-32415-4_20.
- [73] What is a collection offer?, <https://support.opensea.io/hc/en-us/articles/6851928046355-What-is-a-collection-offer->, 2022.
- [74] Collection offer, <https://docs.x2y2.io/guides/how-to-buy-nfts/collection-offer>, 2022.
- [75] Making a collection offer, <https://docs.looksrare.org/guides/buying-nfts/making-a-collection-offer>, 2022.
- [76] Uniswap official website, <https://uniswap.org/>, 2023.
- [77] Nft flashloan — ape coin, <https://medium.com/coinmonks/nft-flashloan-ape-coin-d6065c7dd956>, 2022.
- [78] J. Huang, P. Xia, J. Li, et al., Unveiling the paradox of NFT prosperity, in: Proceedings of the ACM on Web Conference 2024, 2024, pp. 167–177, <https://doi.org/10.1145/3589334.3645566>.
- [79] K. Qin, L. Zhou, B. Livshits, et al., Attacking the defi ecosystem with flash loans for fun and profit, in: International Conference on Financial Cryptography and Data Security, Springer, Berlin, Heidelberg, 2021, pp. 3–32, https://doi.org/10.1007/978-3-662-64322-8_1.
- [80] Animoon rug pull, <https://twitter.com/zachxbt/status/1672032219156688897>, 2022.
- [81] Animoon official website, www.animoon.io, 2023.
- [82] Explained: the animoon rug pull, <https://www.halborn.com/blog/post/explained-the-animoon-rug-pull-june-2022>, 2022.
- [83] Animoon opensea, <https://opensea.io/collection/theanimoont?tab=items>, 2022.
- [84] Nft exchange sudorare goes dark after \$820,000 rug pull, <https://cryptobriefing.com/nft-exchange-sudorare-goes-dark-820000-rug-pull/>, 2022.
- [85] J. Huang, N. He, K. Ma, et al., Miracle or mirage? a measurement study of NFT rug pulls, in: Proceedings of the ACM on Measurement and Analysis of Computing Systems, ACM, 2023, pp. 1–25, <https://doi.org/10.1145/3626782>.
- [86] The gray market: How a brazen hack of that \$69 million beeples revealed the true vulnerability of the nft market (and other insights), <https://news.artnet.com/opinion/sleepminting-nftheft-monsieur-personne-196074>, 2021.
- [87] Rarible, inc., <https://rarible.com/>, 2020.
- [88] B. Guidi, A. Michienzi, Sleepminting, the brand new frontier of Non Fungible Tokens fraud, in: Proceedings of the 2022 ACM Conference on Information Technology for Social Good, ACM, 2022, pp. 75–81, <https://doi.org/10.1145/3524458.3547239>.
- [89] B. Guidi, A. Michienzi, Delving NFT vulnerabilities, a sleepminting prevention system, in: B. Furht (Ed.), Multimedia Tools and Applications, Springer, Cham, 2023, pp. 1–20, <https://doi.org/10.1007/s11042-023-16087-1>.
- [90] P. Xia, H. Wang, B. Gao, et al., Trade or trick? Detecting and characterizing scam tokens on uniswap decentralized exchange, in: Proceedings of the ACM Meas. Anal. Comput. Syst., ACM, 2021, pp. 1–26, <https://doi.org/10.1145/3491051>.
- [91] Moonbirds, <https://opensea.io/collection/proof-moonbirds>, 2023.
- [92] Aja triers tweet, <https://twitter.com/AjaArt/status/1478568569365487616>, 2022.
- [93] F. Wang, Z. Fu, X. Zhang, A self-defense copyright protection scheme for NFT image art based on information embedding, in: Proceedings of ACM Trans. Multimed. Comput. Commun. Appl., ACM, 2024, pp. 1–23, <https://doi.org/10.1145/3652519>.
- [94] New nft wallet-draining exploit-degen meta, <https://medium.com/coinmonks/new-nft-wallet-draining-exploit-degen-meta-f81da02adb6f>, 2022.
- [95] Boomerang, <https://github.com/akamai/boomerang>, 2022.
- [96] Premint official incident briefing, <https://blog.premint.xyz/p/july-17-2022-incident-update>, 2022.
- [97] Analysis of premint malicious code injection, <https://slowmist.medium.com/analysis-of-premint-malicious-code-injection-4051976fd201>, 2022.
- [98] Are nft tools safe? What we can learn from the premint hack?, <https://beosin.com/resources/are-nft-tools-safe-what-we-can-learn-from-the-premint-hack>, 2022.
- [99] Made by apes api information leakage, <https://x.com/zachxbt/status/1691514780119343104>, 2023.
- [100] Opensea nft marketplace hacked for 332 eth, <https://forkast.news/headlines/opensea-nft-marketplace-hacked-332-eth/>, 2022.
- [101] Tal be’ery’s tweet, <https://twitter.com/TalBeerySec/status/1485577572842979329>, 2022.
- [102] Marketplace exploit leads to catastrophic losses for nft collectors, <https://finance.yahoo.com/news/marketplace-exploit-leads-catastrophic-losses-131904566.html>, 2022.
- [103] Get all listings opensea api, https://docs.opensea.io/reference/get_all_listings_on_collection_v2, 2023.
- [104] Bored ape avatar collector loses nfts worth \$567,000 in direct swap scam, <https://indianexpress.com/article/technology/crypto/bored-ape-avatar-collector-loses-nfts-worth-570000-in-direct-swap-scam-7856484/>, 2022.
- [105] Bored ape Holder loses multiples nfts collectively worth \$570,000 in direct swap scam, <https://www.gadgets360.com/cryptocurrency/news/bored-ape-yacht-club-nft-scam-bayc-swapkiwi-exchange-eth-2865694>, 2022.
- [106] Denial-of-service attack, https://en.wikipedia.org/wiki/Denial-of-service_attack, 2024.
- [107] Looksrare, <https://looksrare.org/>, 2024.

- [108] New nft marketplace looksrare launches, suffers ddos attack, <https://finance.yahoo.com/news/nft-marketplace-looksrare-launches-suffers-082959106.html>, 2022.
- [109] Axie ddos, <https://x.com/AxieInfinity/status/1414892981883006977>, 2021.
- [110] Nessus (software), Wikipedia, [https://en.wikipedia.org/wiki/Nessus_\(software\)](https://en.wikipedia.org/wiki/Nessus_(software)), 2023.
- [111] Burp suite, Wikipedia, https://en.wikipedia.org/wiki/Burp_Suite, 2023.
- [112] Slowmist: "blank check" eth_sign phishing analysis, <https://slowmist.medium.com/slow-mist-blank-check-eth-sign-phishing-analysis-741115bd0b1f>, 2022.
- [113] Metamask docs, <https://docs.metamask.io/guide/signing-data.html#a-brief-history>, 2023.
- [114] Erc721 setapprovalforall, <https://docs.openzeppelin.com/contracts/4.x/api/token/erc721#IERC721-setApprovalForAll-address-bool->, 2023.
- [115] Dnstwist, <https://github.com/elceef/dnstwist>, 2023.
- [116] B. He, Y. Chen, Z. Chen, et al., TxPhishScope: towards detecting and understanding transaction-based phishing on ethereum, in: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, ACM, 2023, pp. 120–134, <https://doi.org/10.1145/3576915.3623210>.
- [117] S.S. Roy, D. Das, P. Bose, et al., Unveiling the risks of NFT promotion scams, arXiv: 2301.09806, 2023, preprint, <https://doi.org/10.48550/arXiv.2301.09806>.
- [118] Trojan.phishing, <https://www.f-secure.com/v-descs/trojan-phishing.html>, 2023.
- [119] Scam alert, cthulhuworldp2e, <https://twitter.com/Estetshcrypto/status/1561290861652082689>, 2023.
- [120] Redline stealer malpedia, https://malpedia.caad.fkie.fraunhofer.de/details/win.redline_stealer, 2022.
- [121] How scammers are stealing your crypto with redline malware?, <https://slowmist.medium.com/how-scammers-are-stealing-your-crypto-with-redline-malware-5f246c03bb67>, 2022.
- [122] Spam nfts and how to fix them, <https://www.alchemy.com/overviews/spam-nfts>, 2023.
- [123] Spam nft lets walk, <https://etherscan.io/token/0x9c046c3bc6eb494cb32ef5f17b907b1813dfc3e4>, 2023.
- [124] Forta, <https://forta.org/>, 2023.
- [125] Spam detector, <https://github.com/kovart/forta-spam-detector>, 2023.
- [126] A. Kapoor, D. Guhathakurta, M. Mathur, et al., TweetBoost: influence of social media on NFT valuation, in: Proceedings of the Companion Proceedings of the Web Conference 2022, ACM, 2022, pp. 621–629, <https://doi.org/10.1145/3487553.3524642>.
- [127] Twitter, <https://twitter.com>, 2023.
- [128] Telegram, <https://telegram.org>, 2023.
- [129] Bored ape yacht club discord compromised in \$357,000 nft phishing attack, <https://www.engadget.com/bored-ape-yacht-club-discord-compromised-185957306.html>, 2022.
- [130] Bored ape yacht club's discord server was hacked, with \$360,000 in nfts stolen. Who's to blame is debated, <https://fortune.com/2022/06/04/bored-ape-yacht-clubs-discord-server-was-hacked-with-360000-in-nfts-stolen-blame-debated/>, 2022.
- [131] Z. Wang, J. Gao, X. Wei, Do NFTs' owners really possess their assets? A first look at the NFT-to-asset connection fragility, in: Proceedings of the ACM Web Conference 2023, ACM, 2023, pp. 2099–2109, <https://doi.org/10.1145/3543507.3583281>.
- [132] Liveart official website, <https://liveart.io/NFT>, 2022.
- [133] Q. Wang, R. Li, Q. Wang, et al., Non-fungible token (NFT): overview, evaluation, opportunities and challenges, arXiv:2105.07447, 2021, preprint, <https://doi.org/10.48550/arXiv.2105.07447>.
- [134] P. Praitheeshan, L. Pan, J. Yu, et al., Security analysis methods on ethereum smart contract vulnerabilities: a survey, arXiv:1908.08605, 2019, preprint, <https://doi.org/10.48550/arXiv.1908.08605>.
- [135] M. Saad, J. Spaulding, L. Njilla, et al., Exploring the attack surface of blockchain: a systematic overview, arXiv:1904.03487, 2019, preprint, <https://doi.org/10.48550/arXiv.1904.03487>.
- [136] H. Chen, M. Pendleton, L. Njilla, et al., A survey on ethereum systems security, ACM Comput. Surv. 53 (3) (2021) 1–43, <https://doi.org/10.1145/3391195>.
- [137] S. Werner, D. Perez, L. Gudgeon, et al., SoK: decentralized finance (DeFi), in: Proceedings of the 4th ACM Conference on Advances in Financial Technologies, ACM, 2022, pp. 30–46, <https://doi.org/10.1145/3558535.3559780>.